LAB ASSIGNMENT- 17.3

Task 1:

Use AI assistance to clean an employee information dataset.

Instructions:

- Handle missing values in salary, designation, and experience
- Remove duplicate employee IDs
- Standardize text values (job titles in lowercase)
- Encode categorical features (designation, employment_type)

```python
#Write a Python script using pandas to clean an employee dataset:
#- Fill missing values in salary, designation, experience
#- Remove duplicate employee IDs
#- Convert designation text to lowercase
#- Encode categorical columns (designation, employment_type)

import pandas as pd

# Sample dataset
data = {
    "emp_id": [1, 2, 2, 3],
    "salary": [50000, None, None, 70000],
    "designation": ["Manager", "Engineer", None, "HR"],
    "experience": [5, None, None, 3],
    "employment_type": ["Full-Time", "Part-Time", "Part-Time", "Full-Time"]
}

df = pd.DataFrame(data)

# Fill missing values
df["salary"].fillna(df["salary"].mean(), inplace=True)
df["experience"].fillna(df["experience"].mean(), inplace=True)
df["designation"].fillna("unknown", inplace=True)

# Remove duplicates
df = df.drop_duplicates(subset="emp_id")

# Standardize text
df["designation"] = df["designation"].str.lower()

# Encode categorical data
df = pd.get_dummies(df, columns=["designation", "employment_type"])

print(df)
```

```
   emp_id   salary  experience  designation_engineer  designation_hr  \
0       1  50000.0         5.0                 False           False
1       2  60000.0         4.0                  True           False
3       3  70000.0         3.0                 False            True

   designation_manager  employment_type_Full-Time  employment_type_Part-Time
0                 True                       True                      False
1                False                      False                       True
3                False                       True                      False
/tmp/ipykernel_168/392137802.py:21: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.


  df["salary"].fillna(df["salary"].mean(), inplace=True)
/tmp/ipykernel_168/392137802.py:22: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.


  df["experience"].fillna(df["experience"].mean(), inplace=True)
/tmp/ipykernel_168/392137802.py:23: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.


  df["designation"].fillna("unknown", inplace=True)
```

Task 2 – Student Performance Data Preprocessing

Preprocess student performance data using AI assistance.

Instructions:

- Convert exam_date column into datetime format
- Extract new features (month, semester, day)
- Normalize total_marks
- Detect and handle outliers in marks using IQR or Z-score

```
PROMPT:
Write Python code to preprocess student data:
- Convert exam_date to datetime
- Extract month, day
- Normalize total_marks
- Detect and remove outliers using IQR
```

```python
import pandas as pd

data = {
    "student": ["A", "B", "C", "D"],
    "exam_date": ["2025-01-10", "2025-02-12", "2025-03-15", "2025-04-20"],
    "total_marks": [400, 450, 1000, 420]
}

df = pd.DataFrame(data)

# Convert to datetime
df["exam_date"] = pd.to_datetime(df["exam_date"])

# Extract features
df["month"] = df["exam_date"].dt.month
df["day"] = df["exam_date"].dt.day

# Normalize marks
df["total_marks"] = (df["total_marks"] - df["total_marks"].min()) / (
    df["total_marks"].max() - df["total_marks"].min()
)

# Remove outliers (IQR)
Q1 = df["total_marks"].quantile(0.25)
Q3 = df["total_marks"].quantile(0.75)
IQR = Q3 - Q1

df = df[(df["total_marks"] >= Q1 - 1.5 * IQR) & (df["total_marks"] <= Q3 + 1.5 * IQR)]

print(df)
```

```
  student  exam_date  total_marks  month  day
0       A 2025-01-10     0.000000      1   10
1       B 2025-02-12     0.083333      2   12
3       D 2025-04-20     0.033333      4   20
```

Task 3 – Student Health Data Preparation

Task:

Clean and preprocess student health records using AI scripts.

Instructions:

- Handle missing values in height, weight, BMI
- Encode categorical features (medical_condition, fitness_status)
- Scale numerical features using standard scaling
- Split dataset into training and testing sets

```
PROMPT:
Write Python code to:
- Fill missing values in height, weight, BMI
- Encode categorical columns
- Scale numerical features using StandardScaler
- Split dataset into training and testing sets
```

```python
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

data = {
    "height": [160, 170, None, 180],
    "weight": [60, None, 75, 80],
    "BMI": [22, 24, None, 26],
    "medical_condition": ["None", "Diabetes", "None", "Hypertension"],
    "fitness_status": ["Fit", "Unfit", "Fit", "Fit"]
}

df = pd.DataFrame(data)

# Fill missing values
df.fillna(df.mean(numeric_only=True), inplace=True)

# Encode categorical
df = pd.get_dummies(df, columns=["medical_condition", "fitness_status"])

# Scale
scaler = StandardScaler()
num_cols = ["height", "weight", "BMI"]
df[num_cols] = scaler.fit_transform(df[num_cols])

# Split
X_train, X_test = train_test_split(df, test_size=0.2, random_state=42)

print(X_train)
print(X_test)
```

```
      height    weight       BMI  medical_condition_Diabetes  \
3  1.414214  1.132277  1.414214                       False
0 -1.414214 -1.585188 -1.414214                       False
2  0.000000  0.452911  0.000000                       False

   medical_condition_Hypertension  medical_condition_None  fitness_status_Fit  \
3                            True                   False                True
0                           False                    True                True
2                           False                    True                True

   fitness_status_Unfit
3                 False
0                 False
2                 False
   height  weight  BMI  medical_condition_Diabetes  \
1     0.0     0.0  0.0                        True

   medical_condition_Hypertension  medical_condition_None  fitness_status_Fit  \
1                           False                   False                False

   fitness_status_Unfit
1                  True
```

Task 4: Real-Time Application: Ride-Sharing Data Cleaning

Scenario:

A ride-sharing company has trip data with errors.Requirements:

- Standardize pickup and drop locations
- Fill missing fare amounts with median values
- Remove duplicate ride entries
- Normalize driver ratings
- Generate a summary of data improvements

```
Write Python code to clean ride-sharing data:
- Standardize pickup and drop locations
- Fill missing fare with median
- Remove duplicates
- Normalize driver ratings
- Show summary statistics
```

```python
import pandas as pd

data = {
    "ride_id": [1, 2, 2, 3],
    "pickup": ["chennai", "chennai", "chennai", "Bangalore"],
    "drop": ["Airport", "airport", "airport", "Mall"],
    "fare": [200, None, None, 500],
    "rating": [4.5, 5.0, 5.0, 3.5]
}

df = pd.DataFrame(data)

# Standardize text
df["pickup"] = df["pickup"].str.lower()
df["drop"] = df["drop"].str.lower()

# Fill missing fare
df["fare"].fillna(df["fare"].median(), inplace=True)

# Remove duplicates
df = df.drop_duplicates()

# Normalize ratings
df["rating"] = (df["rating"] - df["rating"].min()) / (
    df["rating"].max() - df["rating"].min()
)

# Summary
print("Clean Data:\n", df)
print("\nSummary:\n", df.describe())
```

```
Clean Data:
   ride_id     pickup     drop   fare    rating
0        1    chennai  airport  200.0  0.666667
1        2    chennai  airport  350.0  1.000000
3        3  bangalore     mall  500.0  0.000000

Summary:
        ride_id   fare    rating
count      3.0    3.0  3.000000
mean       2.0  350.0  0.555556
std        1.0  150.0  0.509175
min        1.0  200.0  0.000000
25%        1.5  275.0  0.333333
50%        2.0  350.0  0.666667
75%        2.5  425.0  0.833333
max        3.0  500.0  1.000000
/tmp/ipykernel_168/2726404960.py:18: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.


  df["fare"].fillna(df["fare"].median(), inplace=True)
```